

コンパイラ演習

WhileLang から WebAssembly への翻訳



今日の到達点

- ▶ コンパイラが行う**段階的な翻訳**を説明できる
- ▶ 構文木を**仮想スタック命令**へ変換できる
- ▶ 命令列を **WebAssembly テキスト形式**へ対応づけられる
- ▶ テストとトレースを使って不具合の場所を絞れる

18 分の流れ

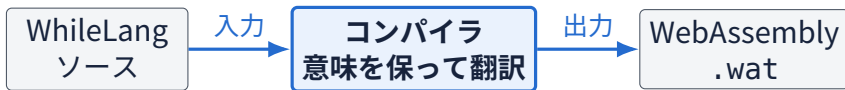
概説 4 分

翻訳例 4 分

課題とヒント 8 分

着手手順 2 分

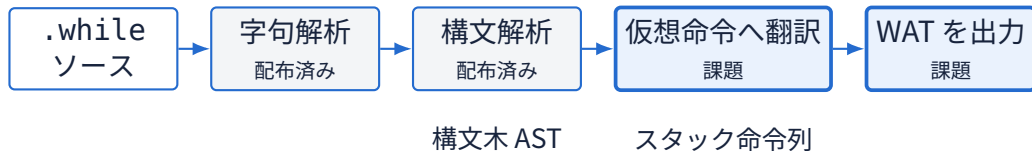
コンパイラの役割



正しさの基準

入力プログラムを直接実行したときと、翻訳後のプログラムを実行したときに、同じ結果が得られること。

今回の翻訳パイプライン



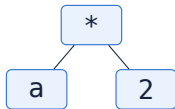
- ▶ Python は `virtual_stack.py` と `emit_wasm.py`
- ▶ C は `virtual_stack.c` と `emit_wasm.c`
- ▶ 最初に一方を選び、途中で混ぜない

式の翻訳は後置記法になる

WhileLang

```
a := a * 2;
```

構文木の考え方



仮想スタック命令

```
rvalue a  
push 2  
*  
lpush a
```

左、右、演算子の順で並べる。演算子はスタック上の二つの値を消費し、結果を一つ積む。

実装の型

```
compile(lhs) + compile(rhs) + [operation]
```

WebAssembly への対応

仮想命令	WAT
push 2	i32.const 2
rvalue a	global.get \$a
lpush a	global.set \$a
+	i32.add
<	i32.lt_s
print	call \$print

Wasm もスタック機械なので、多くの命令はほぼ一対一で対応する。

```
global.get $a
i32.const 2
i32.mul
global.set $a
```

今回の範囲

最適化は行わない。正しい順序で命令を出すことに集中する。

課題 1 算術演算

実装するもの

- ▶ 構文木 Sub / Mul / Div を仮想命令へ翻訳
- ▶ Minus / Times から i32.sub / i32.mul を出力
- ▶ 割り算の WAT 出力 i32.div_s は配布済み

```
case Add(lhs, rhs):  
    return compile_arith(lhs) \  
        + compile_arith(rhs) \  
        + [Plus()]
```

ヒント

Add を写し、最後の命令だけ変える。C では bin_arith を使う。

順序に注意

10 - 3 は 10、3 の順に積む。
逆にすると答えが -7 になる。

課題 2 比較演算

実装する構文木

- ▶ GT, GE, LE, EQ
- ▶ LT が完成例

```
case LT(lhs, rhs):  
    return compile_arith(lhs) \  
        + compile_arith(rhs) \  
        + [Lt()]
```

比較 WAT

比較	WAT
>	i32.gt_s
>=	i32.ge_s
<=	i32.le_s
==	i32.eq

真は 1、偽は 0。 `print i > 1;` のように確認できる。

課題 3 while の命令列

```
while i < 10 do  
  i := i + 1;
```

1. 条件を見る場所を置く
2. 条件が偽なら出口へ飛ぶ
3. 本体を実行する
4. 条件へ戻る
5. 出口を置く

```
label    L.0          # test  
rvalue i  
push    10  
<  
gofalse L.1          # out  
rvalue i  
push    1  
+  
lpush   i  
goto    L.0  
label   L.1
```

while と WAT の構造

```
(block $L.1          ;; out
  (loop $L.0          ;; test
    ... condition ...
    i32.eqz
    br_if $L.1
    ... body ...
    br $L.0
  )
)
```

ラベルは二つ

test: 条件判定へ戻る先

out: ループから抜ける先

- ▶ 入れ子ごとに `gen_label()` で新しい名前を取る
- ▶ `block` / `loop` の出力処理は配布済み

デバッグは手前から確認する

1. **構文木**を見る `python3 whilec.py --ast ../test/arith.while`
2. **仮想命令列**を見る `python3 whilec.py --stack ../test/arith.while`
3. **一命令ずつ実行**する `python3 whilei.py --trace ../test/arith.while`
4. **全テスト**を実行する `python3 test_whilelang.py`

切り分けの原則

AST が正しければパーサを疑わない。WAT の前に `whilei` を通し、仮想命令の誤りを先に直す。

着手順と完了条件

1. Python または C を選ぶ
2. 配布状態で `assign.while` が 3 を出すことを確認
3. 課題 1、課題 2、課題 3 の順に実装
4. 各段階で対応するテストを実行

主な期待値

```
arith.while  
7 8 4 23
```

```
cmp.while  
1 1 0 1 1 0
```

```
simple_loop.while  
10
```

```
fact.while  
120
```

まとめと発展課題

- ▶ 構文木の子を順に翻訳し、最後に演算命令を置く
- ▶ 仮想スタック命令と WAT は多くの場合、一対一で対応する
- ▶ while は test と out の二つのラベルで表す
- ▶ 不具合は AST、命令列、実行トレースの順で切り分ける

時間が余ったら

if P then S else S を、条件、IfStart、then 節、ElseOp、else 節、IfEnd の順で実装する。